

Introduction

Behavioral design patterns are focused on how objects communicate and interact with one another, providing solutions to complex communication scenarios. These patterns help reduce dependencies between components and promote loose coupling, resulting in systems that are easier to modify and scale.

Imagine a scenario where multiple components (e.g., buttons, text fields, and labels) within a user interface need to interact with each other. Instead of directly communicating with each other, the components rely on a central mediator that facilitates communication between them. This decouples the components, making the system more maintainable and flexible. This is the essence of the **Mediator Pattern**.

In the upcoming sections, we will explore the **Mediator Pattern** in detail, understanding how it centralizes communication and reduces the complexity of interactions between objects within a system.

Mediator Pattern

Formal Definition

The **Mediator Pattern** is a behavioral design pattern that centralizes complex communication between objects into a single mediation object. It promotes loose coupling and organizes the interaction between components.

Instead of objects communicating directly with each other, they interact through the mediator, which helps simplify and manage their communication.

Real-Life Analogy: Air Traffic Control (ATC)

In an airport, multiple airplanes communicate with the air traffic control (ATC) tower instead of directly with each other. The ATC coordinates their movements, ensuring safe distances and smooth operations. This simplifies communication, as planes rely on the ATC to manage the flow of information, just like the Mediator Pattern centralizes communication between objects in a system.

Let's now understand the working of **Mediator Pattern** through the help of a problem statement.

Understanding the Problem

Let's imagine a collaborative document editor where users can make changes to a shared document. Each user has the ability to give access to other users, enabling them to collaborate on the same document. The following code snippet demonstrates how this functionality might be implemented:

```
#include <iostream>
```

```
#include <vector>
```

```
#include <string>
```

```
using namespace std;
```

```
// Class representing a User in a collaborative document editor

class User {

public:

    string name;

    vector<User*> others; // List of users that have access to this user


    // Constructor for creating a User with a name
    User(string name) {

        this->name = name;

    }


    // Method to add a collaborator to this user (grants access to the user)
    void addCollaborator(User* user) {

        others.push_back(user);

    }


    // Method to make a change to the document and notify all
    collaborators

    void makeChange(string change) {

        cout << name + " made a change: " + change << endl;

        for (User* u : others) {
```

```
        u->receiveChange(change, this); // Notify each collaborator about
the change
```

```
    }
```

```
}
```

```
// Method to receive a change notification from another user
```

```
void receiveChange(string change, User* from) {
```

```
    cout << name + " received: \" + change + "\" from " + from->name <<
endl;
```

```
}
```

```
};
```

```
// Client Code
```

```
int main() {
```

```
    // Creating users
```

```
    User alice("Alice");
```

```
    User bob("Bob");
```

```
    User charlie("Charlie");
```

```
// Adding collaborators (Alice gives access to Bob and Charlie)
```

```
alice.addCollaborator(&bob);
```

```
alice.addCollaborator(&charlie);
```

```

// Alice makes a change, notifying Bob and Charlie
alice.makeChange("Updated the document title");

// Bob makes a change, notifying Alice and Charlie
bob.makeChange("Added a new section to the document");

return 0;
}

import java.util.*;

// Class representing a User in a collaborative document editor.
class User {

    private String name;

    private List<User> others; // List of users that have access to this user

    // Constructor for creating a User with a name.
    public User(String name) {

        this.name = name;

        this.others = new ArrayList<>();
    }
}

```

```
}
```

```
// Method to add a collaborator to this user (grants access to the user).
```

```
public void addCollaborator(User user) {
```

```
    others.add(user);
```

```
}
```

```
// Method to make a change to the document and notify all  
collaborators.
```

```
// Each collaborator will receive the change notification.
```

```
public void makeChange(String change) {
```

```
    System.out.println(name + " made a change: " + change);
```

```
    for (User u : others) {
```

```
        u.receiveChange(change, this); // Notify each collaborator about  
the change.
```

```
    }
```

```
}
```

```
// Method to receive a change notification from another user.
```

```
public void receiveChange(String change, User from) {
```

```
    System.out.println(name + " received: \"" + change + "\" from " +  
from.name);
```

```
}  
}
```

```
// Client Code
```

```
class Main {
```

```
    public static void main(String[] args) {
```

```
        // Creating users
```

```
        User alice = new User("Alice");
```

```
        User bob = new User("Bob");
```

```
        User charlie = new User("Charlie");
```

```
        // Adding collaborators (Alice gives access to Bob and Charlie)
```

```
        alice.addCollaborator(bob);
```

```
        alice.addCollaborator(charlie);
```

```
        // Alice makes a change, notifying Bob and Charlie
```

```
        alice.makeChange("Updated the document title");
```

```
        // Bob makes a change, notifying Alice and Charlie
```

```
        bob.makeChange("Added a new section to the document");
```

```
}  
}
```

Class representing a User in a collaborative document editor

class User:

def __init__(self, name):

self.name = name

self.others = [] # List of users that have access to this user

Method to add a collaborator to this user (grants access to the user)

def addCollaborator(self, user):

self.others.append(user)

**# Method to make a change to the document and notify all
collaborators**

def makeChange(self, change):

print(f'{self.name} made a change: {change}')

for u in self.others:

**u.receiveChange(change, self) # Notify each collaborator about
the change**


```
# Method to receive a change notification from another user
```

```
def receiveChange(self, change, from_user):
```

```
    print(f"{self.name} received: \"{change}\" from {from_user.name}")
```

```
# Client Code
```

```
if __name__ == "__main__":
```

```
    # Creating users
```

```
    alice = User("Alice")
```

```
    bob = User("Bob")
```

```
    charlie = User("Charlie")
```

```
# Adding collaborators (Alice gives access to Bob and Charlie)
```

```
alice.addCollaborator(bob)
```

```
alice.addCollaborator(charlie)
```

```
# Alice makes a change, notifying Bob and Charlie
```

```
alice.makeChange("Updated the document title")
```

```
# Bob makes a change, notifying Alice and Charlie
```

```
bob.makeChange("Added a new section to the document")
```

Explanation of The Code:

- **addCollaborator(User user):**
This method allows a user to give access to another user, adding them to the list of collaborators.
- **makeChange(String change):**
This method allows the user to make a change to the document. It notifies all the collaborators by calling the receiveChange method on each of them.
- **receiveChange(String change, User from):**
This method is called to notify a user about a change made by another user. It prints out the change and the name of the user who made it.

Issues with the Current Approach

- **Tight Coupling Between Users:**
Each user has references to every other user they collaborate with, creating a tight coupling. This makes it difficult to manage the system when changes (like adding/removing users) need to be made.
- **Adding/Removing Users Breaks the Structure:**
Modifying the list of collaborators (adding or removing users) can easily break the structure, especially in larger systems where users are dynamically managed. This increases the complexity of maintaining the system.

- **Hard to Orchestrate Roles (Editor/Viewer/Admin):**

The current design does not account for different roles (e.g., editor, viewer, admin). Managing these roles within the existing structure would require significant changes, violating the Open-Closed Principle and making the system hard to scale.
 - **Difficulty in Managing Permissions, States, and Notifications:**

The current approach makes it difficult to manage user-specific permissions (e.g., read-only or full access) and notifications (e.g., user roles influencing change notifications). A single user's changes are broadcasted to all collaborators, which makes it challenging to customize behavior based on user roles or states.
 - **Lack of Separation of Concerns:**

The **User class** is responsible for managing collaborators, making changes, and notifying collaborators. This violates the **Single Responsibility Principle (SRP)** as the class is handling multiple responsibilities (collaboration management, change notifications, etc.).
 - **Scalability Issues:**

As the number of users increases, the system becomes harder to manage and maintain due to the direct references between users. The complexity grows rapidly with the addition of new features or users.
-

The Solution

The current implementation of the collaborative document editor can be improved by refactoring the code using the **Mediator Pattern**. Instead of having users directly communicate with each other to notify changes, the **CollaborativeDocument** will act as the mediator. This way, users only interact with the document (mediator) to communicate changes, promoting loose coupling and simplifying the overall structure.

Code:

```
#include <iostream>
```

```
#include <vector>
```

```
#include <string>
```

```
using namespace std;
```

```
// Forward declaration of User class
```

```
class User;
```

```
// Mediator Interface
```

```
class DocumentSessionMediator {
```

```
public:
```

```
    virtual void broadcastChange(string change, User* sender) = 0;
```

```
    virtual void join(User* user) = 0;
```

```
    virtual ~DocumentSessionMediator() {}
```

```
};
```

```
// User Class
```

```
class User {
```

```
protected:
```

```
string name;
```

```
DocumentSessionMediator* mediator;
```

```
public:
```

```
    User(string name, DocumentSessionMediator* mediator) :  
    name(name), mediator(mediator) {}
```

```
// Method for users to make a change
```

```
void makeChange(string change) {
```

```
    cout << name + " edited the document: " + change << endl;
```

```
    mediator->broadcastChange(change, this);
```

```
}
```

```
// Method to receive a change from another user
```

```
void receiveChange(string change, User* sender) {
```

```
    cout << name + " saw change from " + sender->name + ": \" +  
change + "\" << endl;
```

```
}
```

```
};
```

```
// Concrete Mediator Class
```

```
class CollaborativeDocument : public DocumentSessionMediator {
```

private:

```
vector<User*> users;
```

public:

```
void join(User* user) override {
```

```
    users.push_back(user);
```

```
}
```

```
void broadcastChange(string change, User* sender) override {
```

```
    for (User* user : users) {
```

```
        if (user != sender) {
```

```
            user->receiveChange(change, sender);
```

```
        }
```

```
    }
```

```
}
```

```
};
```

```
// Client Code
```

```
int main() {
```

```
    CollaborativeDocument doc;
```

```
// Creating users

User alice("Alice", &doc);

User bob("Bob", &doc);

User charlie("Charlie", &doc);


// Joining the collaborative document

doc.join(&alice);

doc.join(&bob);

doc.join(&charlie);


// Users making changes

alice.makeChange("Added project title");

bob.makeChange("Corrected grammar in paragraph 2");


return 0;

}


import java.util.*;


// Mediator Interface

interface DocumentSessionMediator {
```

```
void broadcastChange(String change, User sender);  
  
void join(User user);  
  
}
```

// Concrete Mediator Class

```
class CollaborativeDocument implements DocumentSessionMediator {  
  
    private List<User> users = new ArrayList<>();
```

```
    @Override
```

```
    public void join(User user) {  
  
        users.add(user);  
  
    }
```

```
    @Override
```

```
    public void broadcastChange(String change, User sender) {  
  
        for (User user : users) {  
  
            if (user != sender) {  
  
                user.receiveChange(change, sender);  
  
            }  
  
        }  
  
    }
```



```
}
```

```
// User Class
```

```
class User {
```

```
    protected String name;
```

```
    protected DocumentSessionMediator mediator;
```

```
    public User(String name, DocumentSessionMediator mediator) {
```

```
        this.name = name;
```

```
        this.mediator = mediator;
```

```
    }
```

```
// Method for users to make a change
```

```
    public void makeChange(String change) {
```

```
        System.out.println(name + " edited the document: " + change);
```

```
        mediator.broadcastChange(change, this);
```

```
    }
```

```
// Method to receive a change from another user
```

```
    public void receiveChange(String change, User sender) {
```

```
        System.out.println(name + " saw change from " + sender.name + ": \"" + change + "\"");
```

```
}  
  
}  
  
// Client Code  
  
class Main {  
  
    public static void main(String[] args) {  
  
        CollaborativeDocument doc = new CollaborativeDocument();  
  
        // Creating users  
  
        User alice = new User("Alice", doc);  
  
        User bob = new User("Bob", doc);  
  
        User charlie = new User("Charlie", doc);  
  
        // Joining the collaborative document  
  
        doc.join(alice);  
  
        doc.join(bob);  
  
        doc.join(charlie);  
  
        // Users making changes  
  
        alice.makeChange("Added project title");  
  
        bob.makeChange("Corrected grammar in paragraph 2");  
  
    }  
}
```

```
}  
}
```

Mediator Interface

class DocumentSessionMediator:

```
    def broadcastChange(self, change, sender):  
        pass
```

```
    def join(self, user):  
        pass
```

Concrete Mediator Class

class CollaborativeDocument(DocumentSessionMediator):

```
    def __init__(self):  
        self.users = []  
  
    def join(self, user):  
        self.users.append(user)
```

```
def broadcastChange(self, change, sender):
```

```
    for user in self.users:
```

```
        if user != sender:
```

```
            user.receiveChange(change, sender)
```

```
# User Class
```

```
class User:
```

```
    def __init__(self, name, mediator):
```

```
        self.name = name
```

```
        self.mediator = mediator
```

```
# Method for users to make a change
```

```
def makeChange(self, change):
```

```
    print(f"{self.name} edited the document: {change}")
```

```
    self.mediator.broadcastChange(change, self)
```

```
# Method to receive a change from another user
```

```
def receiveChange(self, change, sender):
```

```
    print(f"{self.name} saw change from {sender.name}: \"{change}\"")
```

Client Code

```
if __name__ == "__main__":
```

```
    doc = CollaborativeDocument()
```

Creating users

```
alice = User("Alice", doc)
```

```
bob = User("Bob", doc)
```

```
charlie = User("Charlie", doc)
```

Joining the collaborative document

```
doc.join(alice)
```

```
doc.join(bob)
```

```
doc.join(charlie)
```

Users making changes

```
alice.makeChange("Added project title")
```

```
bob.makeChange("Corrected grammar in paragraph 2")
```

Explanation of Changes

- **Mediator Interface (`DocumentSessionMediator`):**
Defines the methods for broadcasting changes and adding users to the document.
- **Concrete Mediator (`CollaborativeDocument`):**
Implements the `DocumentSessionMediator` and manages users. It broadcasts changes to all users except the sender.
- **User Class:**
Now interacts only with the `CollaborativeDocument` (mediator) rather than directly notifying other users. This reduces the coupling between users.

Let's now understand how the Mediator pattern solves the previously discussed issues.

How the Mediator Pattern Solves the Issues

Issue	How it is Solved
Tight Coupling Between Users	The users no longer hold references to each other. They communicate through the mediator (<code>CollaborativeDocument</code>), reducing direct dependencies between them.
Adding/Removing Users Breaks the Structure	The <code>CollaborativeDocument</code> now manages the users and their interactions. Adding or removing users is handled centrally, making the system more maintainable.

Hard to Orchestrate Roles (Editor/Viewer/Admin)	Roles can now be managed through the mediator. Different roles and permissions can be introduced in the CollaborativeDocument class, making the structure more flexible.
Difficulty in Managing Permissions, States, and Notifications	The mediator centralizes the notifications and can be extended to manage permissions, states, and notifications more efficiently.
Lack of Separation of Concerns	The User class now only handles user-specific behavior (making changes and receiving changes), while the mediator handles all communication, adhering to the Single Responsibility Principle .
Scalability Issues	With the mediator handling communication, it is easier to scale the system, as new users or features (like different types of notifications) can be added without altering the existing structure.

When to Use the Mediator Pattern

The Mediator Pattern is most useful in scenarios where multiple objects need to communicate with each other, but direct communication between them would lead to high complexity or tight coupling. Consider using the Mediator Pattern in the following situations:

- **Multiple Users or Services Interacting, but Should Remain Decoupled:**

When you have several users or services that need to interact, but you want to avoid direct dependencies between them, the Mediator Pattern is ideal. By centralizing communication through a mediator, you ensure that the users can communicate without knowing the details of each other's existence or operations.

- **Managing Rules or Permissions Centrally:**

If you need to manage rules or permissions (such as access control or user roles) across multiple components or users, the Mediator Pattern allows you to centralize the logic, making it easier to enforce consistent rules and permissions without modifying each component.

- **Flexible Broadcasting, Filtering, or Transformation of Messages:**

The Mediator Pattern helps when you need to broadcast messages, filter messages, or transform them before they're delivered. By using a mediator, you can introduce flexibility, allowing for dynamic filtering or transformation without affecting the communicating components directly.

Advantages and Disadvantages of Mediator Pattern

Pros

- **Users Don't Need to Know About Other Users:**

The Mediator Pattern decouples the users or components, meaning that they only communicate with the mediator rather than directly with each other. This reduces dependencies and makes the system more flexible and maintainable.

- **Easy to Manage User Roles and Access Centrally:**

Managing user roles (such as admin, editor, and viewer) and permissions becomes easier with the mediator. The mediator can

handle access control centrally without modifying the individual users, ensuring a consistent rule enforcement across the system.

- **Easier to Test and Extend:**

Since the communication between users is centralized in the mediator, testing and extending the system becomes simpler. You can modify or add new users or operations without affecting the entire system.

- **Clean Separation of Business Logic and Interaction:**

The mediator centralizes the logic for interactions between components, which results in a clean separation of concerns. Business logic can be handled separately from the interaction logic, making the system easier to understand and maintain.

Cons

- **Mediator Can Become Complex Over Time:**

As the system grows and more components are added, the mediator may become complex and difficult to maintain. The mediator can end up managing too many responsibilities, making the system harder to scale and debug.

- **One Point of Failure:**

Since the mediator is responsible for handling communication between all components, it becomes a single point of failure. If the mediator encounters issues, it can affect the entire system, leading to potential downtime or failure in communication.

- **Adds an Abstraction Layer:**

Introducing the mediator adds an extra abstraction layer, which may make the system more difficult to understand for developers who are unfamiliar with the pattern. While this abstraction is useful for decoupling components, it can also add complexity to simpler systems.

Real Life Use Cases

The Mediator Pattern is highly useful in systems where multiple components interact, but their communication needs to be centralized or streamlined. Here are two real-world examples of where this pattern can be applied effectively:

1. Airline Management System:

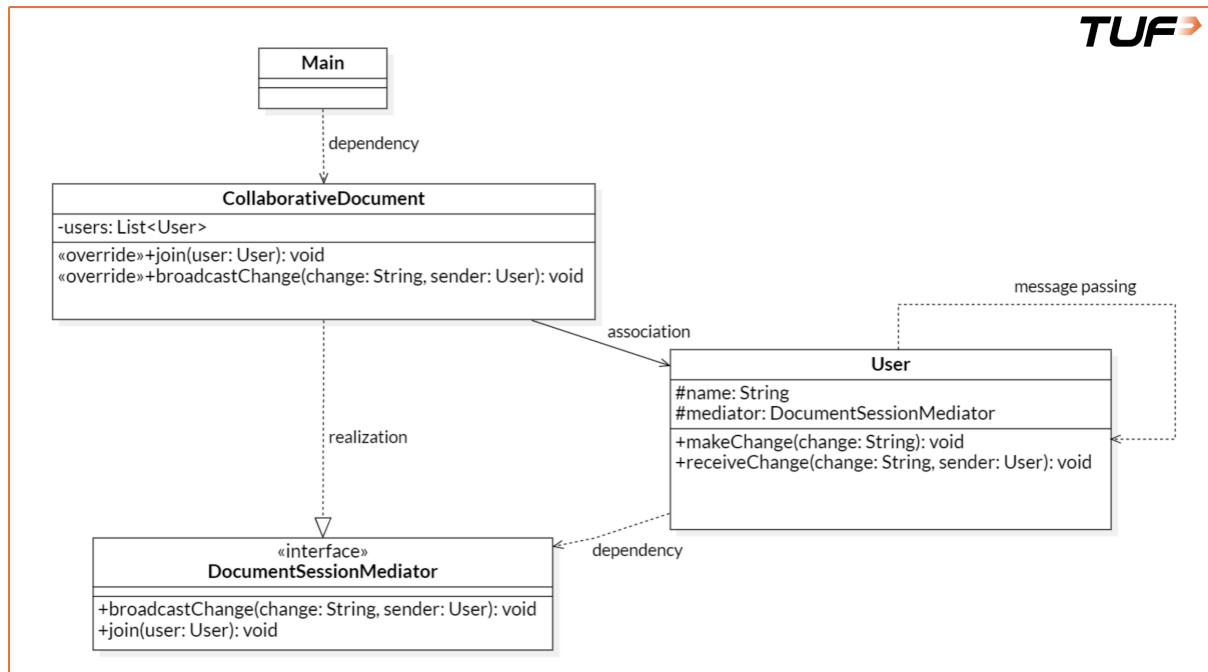
In an airline management system, multiple services (such as booking, customer service, flight status, and payment) need to communicate with each other. Rather than each service directly interacting with the others, a mediator can handle the communication. The mediator ensures that services only communicate with it, reducing dependencies and making the system more maintainable. For instance, when a flight status is updated, the mediator ensures that the booking service, customer service, and payment service all receive the updated information without directly connecting these services.

2. Auction System:

An auction system involves multiple users (bidders) and the auctioneer. Instead of each bidder interacting with every other bidder, the auctioneer can act as a mediator. When a bid is placed, the auctioneer broadcasts the update to all participants. The mediator ensures that all the participants are notified about the bid changes in real-time, ensuring smooth communication and coordination during the auction process.

These examples demonstrate how the Mediator Pattern can simplify complex interactions and centralize control, making the system more flexible and easier to maintain.

Class Diagram



State Pattern - Theory - TUF+